

Lab 4: Data Types and OS Memory Management

Lab Report and Knowledge Check

I. Introduction

This report documents the practical work completed for Lab 4 of ST5003CMD Operating Systems, Security and Networks. A data type is a contract between the programmer, the compiler and the operating system: it fixes how many bytes are allocated, how those bytes are interpreted, and which operations are valid. The lab used two small C programs to observe this relationship directly. The first program uses `sizeof()` to report how much memory each primary data type occupies on this system. The second declares variables that belong to different memory segments (Data, BSS, Stack and Heap) and prints their addresses with `%p`, so the layout of a process's virtual address space can be checked against the lecture material. Both programs were written in nano, compiled with gcc, and run from the `~/lab4` directory.

II. Lab 1 — Data Type Size Exploration

A. The Program

The program `lab1_sizes.c` prints the size, in bytes, of char, int, unsigned int, short, long, long long, float, double and a pointer (using `int*` as the example). The result of `sizeof()` has type `size_t`, so it is printed with the `%zu` format specifier. A pointer is measured with `sizeof(int*)` because every data pointer has the same width on a given architecture.

```
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("short: %zu bytes\n", sizeof(short));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("pointer: %zu bytes\n", sizeof(int*));

    return 0;
}
```

```
abhiyan@nepal: ~/lab4 — nano lab1_sizes.c
GNU nano 8.7.1 lab1_sizes.c *
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");
    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("short: %zu bytes\n", sizeof(short));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("pointer: %zu bytes\n", sizeof(int*));
}
```

Figure 1. lab1_sizes.c opened in nano.

B. Compiling and Running

The program was compiled with gcc into an executable named lab1_sizes. The compiler printed nothing, which means the compilation finished with no errors or warnings, and the executable was then run from the same directory.

```
gcc lab1_sizes.c -o lab1_sizes
./lab1_sizes
```

```
abhiyan@nepal: ~/lab4
cd..: command not found
abhiyan@nepal:~/LabRepo$ cd ..
abhiyan@nepal:~$ mkdir lab4
cd lab4
abhiyan@nepal:~/lab4$ nano lab1_sizes.c
abhiyan@nepal:~/lab4$ gcc lab1_sizes.c -o lab1_sizes
./lab1_sizes
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
unsigned int: 4 bytes
short: 2 bytes
long: 8 bytes
long long: 8 bytes
float: 4 bytes
double: 8 bytes
pointer: 8 bytes
abhiyan@nepal:~/lab4$
```

Figure 2. Compiling lab1_sizes.c without errors and running the program.

C. Comparison with Table I

The output was compared with the 64-bit column of Table I in the lab handout. Every measured value matches the table, which shows that this system is a 64-bit environment: the pointer is 8 bytes wide and long is 8 bytes.

Table 1. Measured sizes compared with the 64-bit column of Table I

Data Type	Measured (bytes)	Table I, 64-bit (bytes)	Match
char	1	1	Yes
short	2	2	Yes
int	4	4	Yes
unsigned int	4	4 (same as int)	Yes
long	8	8	Yes
long long	8	8	Yes
float	4	4	Yes
double	8	8	Yes
pointer (int*)	8	8	Yes

D. Question: Why might sizeof(long) differ between 32-bit and 64-bit systems?

On a 32-bit system, long is 4 bytes because the machine word and the pointer are 32 bits wide, whereas on a 64-bit system it is 8 bytes (here confirmed by the output long: 8 bytes). The C standard only requires long to be at least 32 bits, so the compiler and operating system choose a width that matches the native word size, which lets long hold a full 64-bit address or a large file offset on 64-bit Linux. In contrast, int stays at 4 bytes on both, and long long is always 8 bytes.

Summary: the sizeof() program compiled cleanly and reported char 1, short 2, int and unsigned int 4, long and long long 8, float 4, double 8 and pointer 8 bytes, all consistent with a 64-bit system.

III. Lab 2 — Memory Segment Address Mapping

A. The Program

The program lab2_memory.c declares one variable for each segment of the process address space and prints the address of each with %p. A static local variable was added as well, because it is stored in the Data segment even though it is declared inside main(). The pointer heap_var is itself a local variable, so the pointer lives on the stack while the memory it points to is on the heap.

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;          // Data segment (initialized global)
int bss_var;                  // BSS segment (uninitialized global)

int main() {
    int local_var = 30;        // Stack segment (local variable)
    static int local_static = 20; // Data (static local)

    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var:      %p (Data Segment)\n", (void *)&global_var);
    printf("bss_var:         %p (BSS Segment)\n", (void *)&bss_var);
    printf("local_var:         %p (Stack Segment)\n", (void *)&local_var);
    printf("local_static:      %p (Data Segment)\n", (void *)&local_static);
    printf("heap_var ptr:      %p (Stack - Pointer Variable)\n",
          (void *)&heap_var);
    printf("heap data:         %p (Heap - Allocated Data)\n",
          (void *)heap_var);

    free(heap_var);
    return 0;
}
```

```

GNU nano 8.7.1 lab2_memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;           // Data segment (initialized global)
int bss_var;                   // BSS segment (uninitialized global)

int main() {
    int local_var = 30;         // Stack segment (local variable)
    static int local_static = 20; // Data segment (static local)

    int *heap_var = (int *)malloc(sizeof(int)); // Heap segment allocation
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var:    %p (Data Segment)\n", (void *)&global_var);
}

```

Figure 3. lab2_memory.c opened in nano (top of the program; the remaining printf lines are below the visible area).

B. Compiling and Running

```
gcc lab2_memory.c -o lab2_memory
./lab2_memory
```

```

unsigned int: 4 bytes
short: 2 bytes
long: 8 bytes
long long: 8 bytes
float: 4 bytes
double: 8 bytes
pointer: 8 bytes
abhiyan@nepal:~/lab4$ nano lab2_memory.c
abhiyan@nepal:~/lab4$ gcc lab2_memory.c -o lab2_memory
./lab2_memory
.....Variable Addresses.....

global_var:    0xbe8e99060010 (Data Segment)
bss_var:       0xbe8e9906001c (BSS Segment)
local_var:     0xffffe31bdb9c (Stack Segment)
local_static:  0xbe8e99060014 (Data Segment)
heap_var ptr:  0xffffe31bdba0 (Stack - Pointer Variable)
heap data:     0xbe8ed0230010 (Heap - Allocated Data)
abhiyan@nepal:~/lab4$

```

Figure 4. Output of lab2_memory showing the address of each variable with its segment label.

C. Address Table with Segment Labels

Table 2. Addresses printed by lab2_memory

Variable	Address	Segment	Declared as
global_var	0xbe8e99060010	Data	Initialized global

Variable	Address	Segment	Declared as
local_static	0xbe8e99060014	Data	Static local
bss_var	0xbe8e9906001c	BSS	Uninitialized global
heap data (*heap_var)	0xbe8ed0230010	Heap	malloc() memory
local_var	0xffffe31bdb9c	Stack	Local variable
heap_var (pointer)	0xffffe31bdba0	Stack	Pointer variable

D. Highest and Lowest Addresses

Highest: the Stack. local_var is at 0xffffe31bdb9c and the pointer variable heap_var is at 0xffffe31bdba0, both close to the top of the address space (0xffff...).

Lowest: the Data segment. global_var at 0xbe8e99060010 is the lowest address printed, followed by local_static (0x...14) and then bss_var (0x...1c) in the BSS, which sits directly after the Data segment. All three lie within 12 bytes of each other, inside the same program image.

E. Stack–Heap Address Difference

Using the stack variable local_var and the heap data address:

```
0xffffe31bdb9c (stack, local_var)
- 0xbe8ed0230010 (heap, *heap_var)
-----
= 0x417112f8db8c = 71,953,905,408,908 bytes (about 65.4 TiB)
```

The heap sits between the program image and the stack, and the huge gap between the heap and the stack is unused virtual address space. In the same run the heap is 0x371d0000 = 924,647,424 bytes (about 882 MiB) above global_var.

F. Observations

The addresses follow the memory layout described in the handout. The Data and BSS variables have the lowest addresses and are grouped together, the heap allocation lies above them, and the stack has the highest addresses, so the stack and heap are separated by a very large unused region. The pointer heap_var lives on the stack (0xffffe31bdba0) while the data it points to lives on the heap (0xbe8ed0230010), which shows that a pointer bridges the two segments. The absolute values differ from the handout's examples (0x7fd..., 0x55a1..., 0x404...) because this system uses address space layout randomisation and position-independent executables, so the base addresses change on every run, but the relative order stays the same.

Summary: the program showed Data and BSS at the lowest addresses, the heap above them and the stack at the top, matching the OS process memory layout.

IV. Knowledge Check Questions

Q: What is a data type in the context of programming and operating systems?

A: A data type is a contract between the programmer, the compiler and the operating system. It specifies how many bytes to allocate for a value, how those bytes are interpreted (integer, character or floating point) and which operations are valid on them. In Lab 1, `sizeof()` showed that `int` is 4 bytes and `double` is 8 bytes on this system.

Q: Name the five main memory segments in a process memory layout.

A: The Text segment (machine code), the Data segment (initialized global and static variables), the BSS segment (uninitialized global and static variables), the Heap (dynamically allocated memory) and the Stack (local variables and function calls).

Q: What is the difference between the Data segment and the BSS segment?

A: The Data segment holds global and static variables that have an explicit initial value, such as `global_var = 150` and `local_static = 20`, so their values are stored in the executable file. The BSS segment holds global and static variables with no initial value, such as `bss_var`. Only its size is recorded in the file, and the OS fills it with zeros when the program is loaded. In Lab 2, `bss_var` (0x...1c) was placed right after the Data variables (0x...10 and 0x...14).

Q: Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

A: Text, Data, BSS and the heap start at the low end and grow upward, while the stack starts at the top and grows downward. This gives the heap and stack the maximum room to expand toward each other before they could collide. Fixed-size regions such as the code are put at the bottom, and the huge gap between them, seen as about 65 TiB in Lab 2, is left free.

Q: What happens to stack memory when a function returns? Who reclaims it?

A: When a function returns, its stack frame (local variables, saved registers and return address) is popped off by moving the stack pointer back, and the space is immediately available for the next call. The compiler-generated code (function prologue and epilogue) does this automatically, so the programmer does nothing. The values are not erased, only marked reusable; the OS reclaims the whole stack only when the process ends.

Q: How does the OS know which memory belongs to which process?

Each process has its own virtual address space and its own page table, which is recorded in the process's control block. The memory management unit (MMU) uses that page table to translate virtual addresses into physical frames, and the OS loads the right page table on each context switch. A process can therefore only reach frames mapped in its own table, which provides isolation between processes.

V. Lab Tasks Completed

1. Wrote `lab1_sizes.c` to print the size of `char`, `int`, `unsigned int`, `short`, `long`, `long long`, `float`, `double` and a pointer, compiled it without errors, and ran it (Figures 1–2).

2. Compared the output with the 64-bit column of Table I and answered why `sizeof(long)` differs between 32-bit and 64-bit systems (Table 1).
3. Wrote `lab2_memory.c` with a global initialized variable, a global uninitialized variable, a local variable, a static local variable and a heap-allocated variable, and printed each address with `%p` (Figures 3–4).
4. Labelled each variable with its memory segment and identified the highest address (Stack) and the lowest address (Data) (Table 2).
5. Calculated the address difference between the stack and the heap (`0x417112f8db8c`, about 65.4 TiB) and wrote the observations.

VI. Conclusion

This lab connected the way data types are declared in C with the way the operating system lays out memory. Lab 1 showed that type sizes are decided by the architecture: on this 64-bit system pointers and long are 8 bytes, while int and float stay at 4. Lab 2 showed that variables are placed in different segments depending on how they are declared: initialized globals and static locals in Data, uninitialized globals in BSS, `malloc()` memory on the heap and local variables on the stack, with the stack at the highest addresses and Data at the lowest. It also showed that a pointer variable sits on the stack while the memory it points to sits on the heap.